

Chapter 1: Pattern Basics & Simple Patterns

SLIDE 1 — WHAT ARE PATTERNS?

Definition

A pattern is a structured arrangement of **characters, numbers, or symbols** printed in a specific order.

In C programming, patterns are usually created using **loops**.

Simple Idea

Rows + Columns + Repetition → Pattern

Example

A simple star pattern:

```
*  
**  
***  
****  
*****
```

Here:

- The pattern has **5 rows**.
- Each row contains stars.
- The number of stars increases from one row to the next.

Understanding a Pattern

Every pattern can be studied using:

1. **Rows** — How many lines are printed?
2. **Columns** — How many elements are printed in each row?
3. **Elements** — What is being printed?
4. **Change** — How does the output change from row to row?

Analogy

Think of arranging chairs in a classroom:

Row 1 → 1 chair
Row 2 → 2 chairs
Row 3 → 3 chairs
Row 4 → 4 chairs

The arrangement follows a pattern.

Remember

Pattern = A repeated and organized arrangement of output.

SLIDE 2 — PATTERN → LOGIC

How Do We Solve a Pattern?

Before writing C code, first **observe the pattern**.

Do not immediately start typing loops. The keyboard has survived long enough.

Step 1 — Count the Rows

Example:

```
*  
**  
***  
****  
*****
```

There are **5 rows**.

Step 2 — Count Elements in Each Row

Row 1 → 1 star
Row 2 → 2 stars
Row 3 → 3 stars
Row 4 → 4 stars
Row 5 → 5 stars

Step 3 — Identify What Changes

The number of stars increases by **1** in every row.

1 → 2 → 3 → 4 → 5

Step 4 — Convert the Logic into Loops

We need:

- One loop to control the **rows**.
- Another loop to control the **stars in each row**.

Simple Flow

Observe Pattern



Count Rows



Count Elements



Find the Change



Create Loop Logic



Print Pattern

Remember

First understand the pattern. Then write the code.

SLIDE 3 — BASIC NESTED LOOP STRUCTURE

What Is a Nested Loop?

A **nested loop** is a loop placed inside another loop.

Patterns commonly use nested loops because we need to:

- Repeat rows.
- Repeat elements inside each row.

Basic Structure

```
for (row = 1; row <= n; row++) {
```

```
    for (column = 1; column <= row; column++) {
```

```
    printf("*");  
}  
  
printf("\n");  
}
```

Two Loops

Outer Loop

Controls the number of rows.

```
for (row = 1; row <= n; row++)
```

Inner Loop

Controls how many elements are printed in each row.

```
for (column = 1; column <= row; column++)
```

How It Works

Outer Loop

↓

Row 1 → Inner Loop → Print elements

↓

Row 2 → Inner Loop → Print elements

↓

Row 3 → Inner Loop → Print elements

↓

...

Why \n?

```
printf("\n");
```

moves the cursor to the **next line** after one row is completed.

Without it, the entire pattern may appear on one line.

Remember

Outer loop = Rows

Inner loop = Elements inside each row

\n = Move to the next row

SLIDE 4 — INCREASING & DECREASING STAR PATTERNS

Increasing Star Pattern

```
*  
**  
***  
****  
*****
```

The number of stars **increases** as the row number increases.

Row 1 → 1 star
Row 2 → 2 stars
Row 3 → 3 stars
Row 4 → 4 stars
Row 5 → 5 stars

C Example

```
#include <stdio.h>
```

```
int main() {  
  
    int row, column;  
  
    for (row = 1; row <= 5; row++) {  
  
        for (column = 1; column <= row; column++) {  
            printf("*");  
        }  
  
        printf("\n");  
    }  
  
    return 0;  
}
```

Decreasing Star Pattern

```
*****  
****  
***  
**  
*
```

Here, the number of stars **decreases** as the row number increases.

Row 1 → 5 stars
Row 2 → 4 stars
Row 3 → 3 stars
Row 4 → 2 stars
Row 5 → 1 star

Main Difference

Increasing → Stars increase
Decreasing → Stars decrease

Important Idea

The **outer loop** controls the row.

The **inner loop** decides how many stars are printed in that row.

Remember

Increasing pattern → More elements each row

Decreasing pattern → Fewer elements each row

SLIDE 5 — COMMON BEGINNER MISTAKES

Mistake 1 — Wrong Loop Limit

Incorrect:

```
for (row = 1; row < 5; row++)
```

This produces only **4 rows**.

Correct:

```
for (row = 1; row <= 5; row++)
```

Mistake 2 — Missing Newline

Incorrect:

```
printf("**");
```

If the newline is missing after each row, the output can become:

```
*****
```

Correct:

```
printf("**");  
printf("\n");
```

Mistake 3 — Incorrect Loop Nesting

The inner loop must be **inside** the outer loop.

```
for (row = 1; row <= 5; row++) {  
  
    for (column = 1; column <= row; column++) {  
        printf("**");  
    }  
  
    printf("\n");  
}
```

Mistake 4 — Printing Outside the Inner Loop

Incorrect:

```
for (column = 1; column <= row; column++) {  
}  
  
printf("**");
```

The star is printed only once instead of once for every inner-loop iteration.

Correct Thinking

Outer Loop



Start Row



Inner Loop



Print Every Element



New Line



Next Row

Remember

When a pattern looks wrong, check:

1. **Loop limits**
2. **Loop nesting**
3. **Inner-loop condition**
4. **Newline position**
5. **Where the `printf()` is placed**

One-Line Recap

Patterns are built by understanding rows, elements, repetition, and using nested loops to reproduce that structure in C.

QUESTIONNAIRES:

1. Theory MCQ — Slide 1: What Are Patterns?

What is a **pattern** in C programming?

- A) A single value stored in a variable
- B) A structured arrangement of characters, numbers, or symbols
- C) A type of data type
- D) A mathematical operator

Answer: B) A structured arrangement of characters, numbers, or symbols ✓

📌 **From: Slide 1 — What Are Patterns?**

💡 **Hint:**

Think about an organized arrangement that is printed in a specific order.

Glossary — pattern

A repeated and organized arrangement of output.

2. Theory MCQ — Slide 2: Pattern → Logic

Before writing loops for a pattern, what should you do first?

- A) Start typing the code
- B) Count the variables
- C) Observe the pattern
- D) Print the output

Answer: C) Observe the pattern ✓

📌 **From: Slide 2 — Pattern → Logic**

💡 **Hint:**

Don't immediately write code. First understand how the pattern is arranged.

3. Code Output MCQ — Slide 3: Basic Nested Loop Structure

What will be the output?

```
int row, column;
```

```
for (row = 1; row <= 3; row++) {  
    for (column = 1; column <= row; column++) {  
        printf("*");  
    }  
    printf("\n");  
}
```

A)

B)

*

**

C)

**

*

D)

*

*

*

Answer: B)

*

**

 **From: Slide 3 — Basic Nested Loop Structure**

 **Hint:**

The outer loop controls the rows, while the inner loop prints stars equal to the current row number.

4. Fill in the Output — Slide 4: Increasing & Decreasing Star Patterns

Fill in the blank with the correct output:

```
int row, column;
```

```
for (row = 1; row <= 3; row++) {  
    for (column = 1; column <= row; column++) {  
        printf("*");  
    }  
    printf("\n");  
}
```

Output:

*

—

Answer: ** 

 **From: Slide 4 — Increasing & Decreasing Star Patterns**

 **Hint:**

In an increasing pattern, the number of stars increases with each row.

5. Find the Mistake — Slide 5: Common Beginner Mistakes

What is wrong with the following code?

```
for (row = 1; row < 5; row++) {  
    printf("*");  
    printf("\n");
```

```
}
```

- A) The `printf()` statement is missing
- B) The loop produces only 4 rows instead of 5
- C) The inner loop is missing
- D) `row++` is incorrect

Answer: B) The loop produces only 4 rows instead of 5 

 **From: Slide 5 — Common Beginner Mistakes**

 **Hint:**

Check the difference between `< 5` and `<= 5`.